

**BỘ KHOA HỌC VÀ CÔNG NGHỆ
HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG
CƠ SỞ THÀNH PHỐ HỒ CHÍ MINH
KHOA CÔNG NGHỆ THÔNG TIN II**

---o0o---



**BÁO CÁO CUỐI KỲ
MÔN: An toàn mạng nâng cao
ĐỀ TÀI: Hệ thống phát hiện đe dọa đa lớp (Packet-
Flow-Session) bằng AI đáp ứng thời gian thực**

Giảng viên	ThS. Đàm Minh Linh	
Sinh viên	Hồ Văn Tâm	N22DCAT046
	Nguyễn Xuân Quang	N22DCAT043
Lớp	D22CQAT01-N	
Khoa	Công nghệ thông tin 2	
Khóa	2022-2027	

TP.HCM, tháng 06 năm 2026

MỤC LỤC

MỤC LỤC	1
DANH SÁCH BẢNG.....	3
DANH SÁCH HÌNH ẢNH	4
LỜI CẢM ƠN.....	5
LỜI NÓI ĐẦU	6
BẢNG PHÂN CHIA CÔNG VIỆC.....	7
CHƯƠNG I. CƠ SỞ LÝ THUYẾT.....	9
1.1. Gói tin	9
1.2. Flow	9
1.3. Session	9
1.4. Đa luồng.....	9
1.5. Xử lý song song	10
1.6. Ensemble (cơ chế hợp nhất quyết định)	10
1.7. Hệ thống phát hiện xâm nhập (IDS).....	10
CHƯƠNG II. PHÂN TÍCH VÀ THIẾT KẾ	11
2.1 Mô hình mạng tổng quát.....	11
2.2. Thiết kế mô hình phát hiện xâm nhập đa tầng.....	11
CHƯƠNG III. CÀI ĐẶT VÀ TRIỂN KHAI	15
3. Cài đặt và triển khai mô hình phát hiện xâm nhập đa tầng	15
CHƯƠNG IV. KIỂM THỬ VÀ ĐÁNH GIÁ.....	23
4.1. Mục tiêu kiểm thử.....	23
4.2. Môi trường kiểm thử.....	23
4.3. Phương pháp kiểm thử.....	23
4.4. Đánh giá offline trên tập test	23
4.5. Kiểm thử live trong môi trường triển khai	25
4.6. Đánh giá độ trễ xử lý	26
4.7. Đánh giá throughput	26
4.8. Đánh giá tài nguyên CPU và RAM	27
4.9. Phân tích kết quả theo từng layer	28

4.10. Hạn chế và hướng cải thiện	29
TÀI LIỆU THAM KHẢO	30

DANH SÁCH BẢNG

Bảng 3.1: Các dataset được sử dụng để huấn luyện mô hình	15
Bảng 3.2: phân bố nhãn.....	16
Bảng 3.3: Khái quát quy trình huấn luyện mô hình.....	18
Bảng 3.4: Thông số mô hình tầng 1	19
Bảng 3.5: Thông số mô hình tầng 2	19
Bảng 3.6: Thông số mô hình tầng 3	20
Bảng 3.7: Các chỉ số đo lường hiệu quả	20
Bảng 3.8: Ma trận rời	20
Bảng 3.9: Ngưỡng cảnh báo cho từng tầng	21
Bảng 3.10: Khoảng thời gian áp dụng khử trùng lặp.....	21
Bảng 3.11: Các luồng chạy song song.....	22
Bảng 4.1: Môi trường.....	23
Bảng 4.2: Các nhóm kiểm thử	23
Bảng 4.3: Metric	24
Bảng 4.4: Kết quả offilne.....	24
Bảng 4.5: Kịch bản kiểm thử	25

DANH SÁCH HÌNH ẢNH

Hình 2.1: Mô hình mạng tổng quát.....	11
Hình 2.2: Kiến trúc tổng quát	11
Hình 2.3: Sơ đồ so sánh vùng quan sát của tầng gói tin, tầng flow và tầng phiên kết nối.....	12
Hình 2.4: Sơ đồ kiến trúc đa luồng gồm T1-Capture, T2-L1Infer, T3-L2Infer, T4-ZeekL3, T5-Alert và cơ chế FlowCleanup.	13
Hình 2.5: Sơ đồ giản lược kiến trúc đa luồng.....	13
Hình 2.6: Sơ đồ luồng dữ liệu thời gian thực từ lưu lượng mạng, qua các tầng phát hiện, đến cảnh báo trên Wazuh.	14
Hình 2.7: Sơ đồ nguyên lý ensemble không đồng nhất và cơ chế hợp nhất cảnh báo ở mức quyết định.....	14
Hình 3.1: Sơ đồ pipeline từ dữ liệu thô, tiền xử lý, huấn luyện đến lưu artifact triển khai.	15
Hình 4.1: Bảng Confusion matrix.....	24
Hình 4.2: Kết quả Live Detection.....	25
Hình 4.3: Độ trễ xử lý.....	26
Hình 4.4: Throughput.....	27
Hình 4.5: Tài nguyên sử dụng.....	27

LỜI CẢM ƠN

Trước hết, chúng em xin gửi lời cảm ơn chân thành đến ThS. Đàm Minh Linh, giảng viên môn an toàn mạng nâng cao, đã tận tình giảng dạy, hướng dẫn và cung cấp những kiến thức nền tảng quan trọng trong suốt quá trình học tập.

Do kiến thức và kinh nghiệm thực hành còn hạn chế, bài báo cáo khó tránh khỏi những thiếu sót trong quá trình trình bày, phân tích và đánh giá kết quả. Chúng em rất mong nhận được sự góp ý của thầy để có thể hoàn thiện hơn về kiến thức chuyên môn cũng như kỹ năng thực hành phân tích và giám sát an toàn mạng trong thời gian tới.

Chúng em xin chân thành cảm ơn!

LỜI NÓI ĐẦU

Trong bối cảnh các hệ thống thông tin ngày càng phát triển, các cuộc tấn công mạng cũng trở nên đa dạng và phức tạp hơn. Một hệ thống phát hiện xâm nhập (IDS) hoạt động đơn lớp luôn có những điểm mù nhất định; ví dụ, nếu chỉ nhìn từng gói tin thì sẽ không thấy được bức tranh toàn cảnh của một cuộc tấn công kéo dài. Do đó, đề tài đưa ra giải pháp "xếp chồng" 3 lớp phân tích trí tuệ nhân tạo (AI) chạy song song, đảm bảo nếu kẻ tấn công lách qua được lớp này thì sẽ bị tóm gọn ở lớp khác.

Đề tài “Hệ thống phát hiện đe dọa đa lớp (Packet–Flow–Session) bằng AI đáp ứng thời gian thực” được xây dựng nhằm giải quyết một hạn chế lớn của các hệ thống an ninh mạng truyền thống: "Vấn đề Điểm mù", gồm ba lớp Packet, Flow và Session.

Báo cáo trình bày cơ sở lý thuyết về mô hình phát hiện xâm nhập 3 lớp; đồng thời triển khai hệ thống thực nghiệm. Do kiến thức và kinh nghiệm thực hành còn hạn chế, báo cáo khó tránh khỏi những thiếu sót. Rất mong nhận được sự góp ý của giảng viên để bài báo cáo được hoàn thiện hơn.

BẢNG PHÂN CHIA CÔNG VIỆC

STT	Hạng mục công việc trọng tâm	Phân công	Chi tiết vai trò
1	Nghiên cứu & Thiết kế Kiến trúc tổng thể	Hồ Văn Tâm	Chịu trách nhiệm thiết kế mô hình Đa lớp (Packet-Flow-Session) và nguyên lý xử lý song song.
2	Thu thập & Tiền xử lý 4 bộ Dữ liệu lớn	Nguyễn Xuân Quang	Gom nhóm, làm sạch, chuẩn hóa nhãn 4 dataset (Firewall, CICIDS2017, UNSW-NB15, Bot-IoT).
3	Xây dựng bộ Đặc trưng (Feature Engineering)	Nguyễn Xuân Quang	Phân tích tương quan, trích xuất 5 đặc trưng cho Lớp 1, 15 đặc trưng cho Lớp 2 và 12 đặc trưng cho Lớp 3.
4	Huấn luyện AI cho TOÀN BỘ 3 Lớp (L1, L2, L3)	Hồ Văn Tâm	Code thuật toán Decision Tree (Lớp 1), LightGBM (Lớp 2 & Lớp 3), cân bằng nhãn (SMOTE), lưu file .pkl.
5	Thiết lập Môi trường & Viết Script tấn công	Nguyễn Xuân Quang	Cài đặt Ubuntu 20.04, Kali Linux. Viết kịch bản kali_attacks.sh với 6 loại tấn công

			(Nmap, Hydra, Hping3).
6	Lập trình Core Engine & Kiến trúc Đa luồng	Hồ Văn Tâm	Code luồng xử lý bất đồng bộ (Threads & Queues: raw_q, flow_q), tích hợp AI vào thời gian thực.
7	Tích hợp Scapy & Zeek vào Pipeline	Nguyễn Xuân Quang	Lập trình module dùng Scapy bắt gói tin mạng và module đọc file conn.log của Zeek cho AI phân tích.
8	Lập trình Cổng kiểm duyệt (EnsembleGate)	Hồ Văn Tâm	Viết thuật toán khử trùng lặp cảnh báo, chặn cảnh báo giả từ luồng phản hồi (Known-Attacker Suppression).
9	Đo lường các thông số Hệ thống (Metrics)	Nguyễn Xuân Quang	Chạy thực tế để đo lường RAM, CPU, Throughput (thông lượng) và Latency (độ trễ).
10	Đánh giá Mô hình & Ma trận rối (Confusion Matrix)	Hồ Văn Tâm	Phân tích sâu các chỉ số F1, Precision, Recall, AUC và ma trận nhầm lẫn của từng lớp AI.
11	Tổng hợp Báo cáo & Slide bảo vệ	Cả hai thành viên	Tổng duyệt nội dung kiến trúc và AI.

CHƯƠNG I. CƠ SỞ LÝ THUYẾT

1.1. Gói tin

Gói tin là đơn vị dữ liệu cơ bản được truyền trên mạng máy tính. Khi một thiết bị gửi thông tin qua mạng, dữ liệu sẽ được chia thành nhiều gói nhỏ để truyền đi, sau đó được bên nhận ghép lại thành dữ liệu hoàn chỉnh. Mỗi gói tin thường bao gồm phần tiêu đề và phần dữ liệu. Phần tiêu đề chứa các thông tin điều khiển như địa chỉ IP nguồn, địa chỉ IP đích, giao thức sử dụng, cổng nguồn, cổng đích và một số cờ điều khiển. Phần dữ liệu chứa nội dung thực tế được truyền tải.

Trong bài toán phát hiện xâm nhập, phân tích gói tin cho phép quan sát trực tiếp các dấu hiệu bất thường xuất hiện ngay từ thời điểm đầu của giao tiếp mạng, chẳng hạn như truy cập vào cổng nhạy cảm, sử dụng cờ TCP bất thường hoặc xuất hiện lưu lượng có kích thước khác thường. Tuy nhiên, gói tin chỉ phản ánh thông tin cục bộ tại một thời điểm, nên thường chưa đủ để mô tả toàn bộ hành vi tấn công kéo dài qua nhiều lần trao đổi.

1.2. Flow

Flow là tập hợp các gói tin có cùng một ngữ cảnh giao tiếp mạng trong một khoảng thời gian nhất định. Trong thực tế, flow thường được xác định dựa trên bộ 5 thuộc tính gồm địa chỉ IP nguồn, địa chỉ IP đích, cổng nguồn, cổng đích và giao thức. Các gói tin có cùng bộ thuộc tính này sẽ được gom lại để hình thành một flow.

Khác với gói tin đơn lẻ, flow biểu diễn được đặc trưng tổng hợp của một quá trình trao đổi dữ liệu, ví dụ như thời lượng truyền, tổng số packet, tổng số byte, số packet theo từng chiều hoặc tốc độ truyền dữ liệu. Nhờ đó, phân tích flow giúp phát hiện tốt hơn các hành vi như scan, brute force, flood hoặc những phiên giao tiếp có mẫu lưu lượng bất thường. Trong hệ thống của đề tài, flow đóng vai trò trung gian giữa mức packet và mức session, cho phép giữ được tốc độ xử lý tương đối cao nhưng vẫn bổ sung thêm ngữ cảnh so với phân tích gói tin đơn lẻ.

1.3. Session

Session là mức quan sát cao hơn flow, phản ánh toàn bộ quá trình thiết lập, duy trì và kết thúc một phiên kết nối giữa hai thực thể trong mạng. Nếu flow chủ yếu mô tả thống kê lưu lượng, thì session còn chứa thêm thông tin về trạng thái kết nối, chiều truyền dữ liệu, hành vi lặp lại và đặc điểm của tiến trình giao tiếp theo thời gian.

Trong giám sát an toàn thông tin, session đặc biệt hữu ích để nhận diện các hành vi phức tạp như brute force kéo dài, kết nối bị từ chối lặp lại, duy trì truy cập trái phép hoặc chuỗi kết nối bất thường đến cùng một dịch vụ. Tuy nhiên, để xây dựng session cần có thêm bước theo dõi và tổng hợp dữ liệu, vì vậy độ trễ xử lý thường cao hơn so với packet và flow. Trong đề tài, session được khai thác thông qua log Zeek conn.log để tạo góc nhìn bổ sung ở tầng phát hiện thứ ba.

1.4. Đa luồng

Đa luồng là kỹ thuật cho phép một chương trình được chia thành nhiều luồng thực thi khác nhau trong cùng một tiến trình. Mỗi luồng có thể đảm nhiệm một công việc riêng, ví dụ như thu thập dữ liệu, tiền xử lý, suy luận mô hình hoặc ghi log. Việc sử dụng đa luồng giúp hệ thống

không bị phụ thuộc vào một luồng xử lý duy nhất và có thể tận dụng tài nguyên tính toán tốt hơn.

Trong bài toán phát hiện xâm nhập thời gian thực, đa luồng có ý nghĩa rất quan trọng vì các bước xử lý thường có tốc độ không đồng đều. Chẳng hạn, bắt packet phải diễn ra liên tục, trong khi phân tích flow chỉ được thực hiện khi flow kết thúc, còn đọc log session lại phụ thuộc vào thời điểm hệ thống ghi log. Nếu tất cả công việc này chạy tuần tự trong một luồng, toàn bộ pipeline rất dễ bị nghẽn. Vì vậy, việc tổ chức hệ thống theo nhiều thread giúp tăng tính ổn định và giảm độ trễ trong quá trình vận hành.

1.5. Xử lý song song

Xử lý song song là phương pháp thực hiện nhiều tác vụ cùng lúc nhằm rút ngắn thời gian xử lý tổng thể và tăng thông lượng hệ thống. Khái niệm này rộng hơn đa luồng, vì nó có thể được thực hiện bằng nhiều luồng trong cùng một tiến trình, nhiều tiến trình độc lập hoặc nhiều thiết bị tính toán khác nhau.

Trong hệ thống phát hiện xâm nhập, xử lý song song cho phép các tầng phát hiện hoạt động đồng thời trên những mức dữ liệu khác nhau. Một packet có thể vừa được đưa vào bộ phân tích packet, vừa được gom vào flow, đồng thời sau đó được hệ thống ghi nhận ở mức session. Nhờ cơ chế xử lý song song, hệ thống có thể phát hiện sớm ở tầng đầu nhưng vẫn tiếp tục khai thác thêm ngữ cảnh ở các tầng sau mà không phải chờ tuần tự từng bước hoàn tất.

1.6. Ensemble (cơ chế hợp nhất quyết định)

Ensemble là phương pháp kết hợp nhiều mô hình hoặc nhiều nguồn quyết định để tạo ra kết quả cuối cùng tốt hơn so với chỉ sử dụng một mô hình đơn lẻ. Ý tưởng chính của ensemble là mỗi mô hình có một điểm mạnh và điểm yếu riêng; khi kết hợp hợp lý, hệ thống có thể giảm sai số, tăng độ ổn định và cải thiện hiệu quả phát hiện.

Trong đề tài này, ensemble không được hiểu theo nghĩa kết hợp nhiều mô hình giống nhau trên cùng một bộ đặc trưng, mà theo nghĩa kết hợp nhiều tầng phát hiện khác nhau. Mỗi tầng quan sát mạng ở một mức riêng, gồm packet, flow và session. Tầng packet cho khả năng cảnh báo sớm, tầng flow cho cái nhìn tổng hợp hơn về lưu lượng, còn tầng session cho thêm ngữ cảnh kết nối. Việc tổng hợp kết quả từ nhiều tầng giúp hệ thống phát hiện tốt hơn các hành vi tấn công đa dạng và hạn chế điểm mù của từng tầng riêng lẻ.

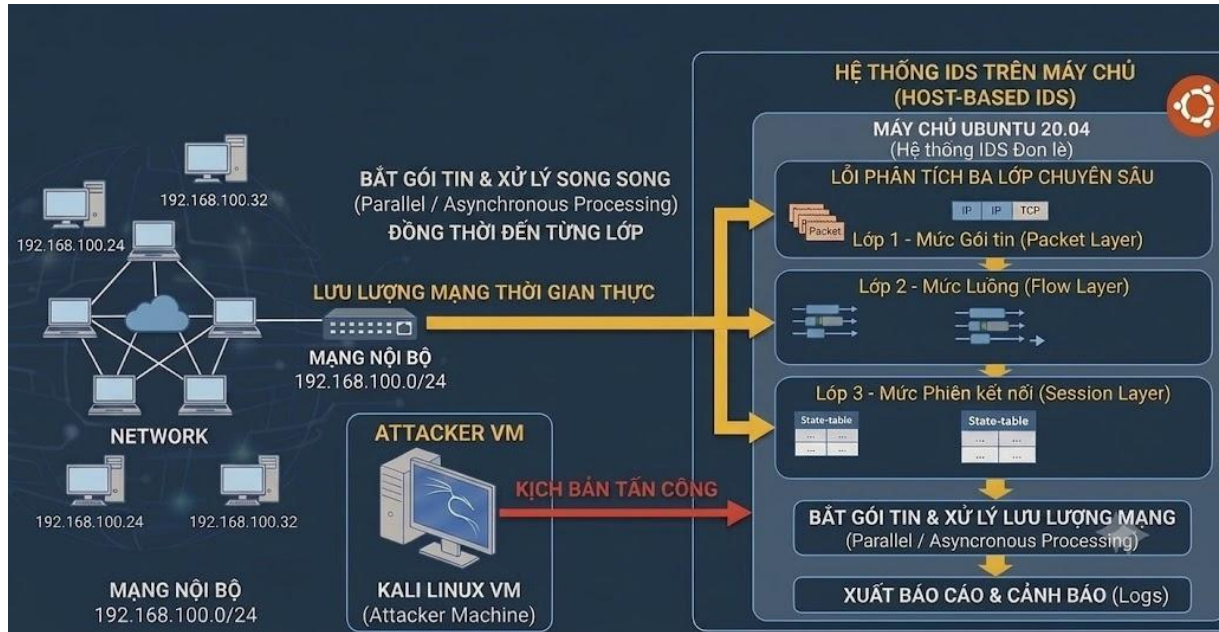
1.7. Hệ thống phát hiện xâm nhập (IDS)

IDS là viết tắt của Intrusion Detection System, tức hệ thống phát hiện xâm nhập. Đây là hệ thống có nhiệm vụ giám sát hoạt động của mạng hoặc máy chủ để phát hiện các dấu hiệu bất thường, hành vi tấn công hoặc vi phạm chính sách an toàn thông tin. Khi phát hiện sự kiện đáng ngờ, IDS sẽ tạo cảnh báo để quản trị viên phân tích và xử lý.

CHƯƠNG II. PHÂN TÍCH VÀ THIẾT KẾ

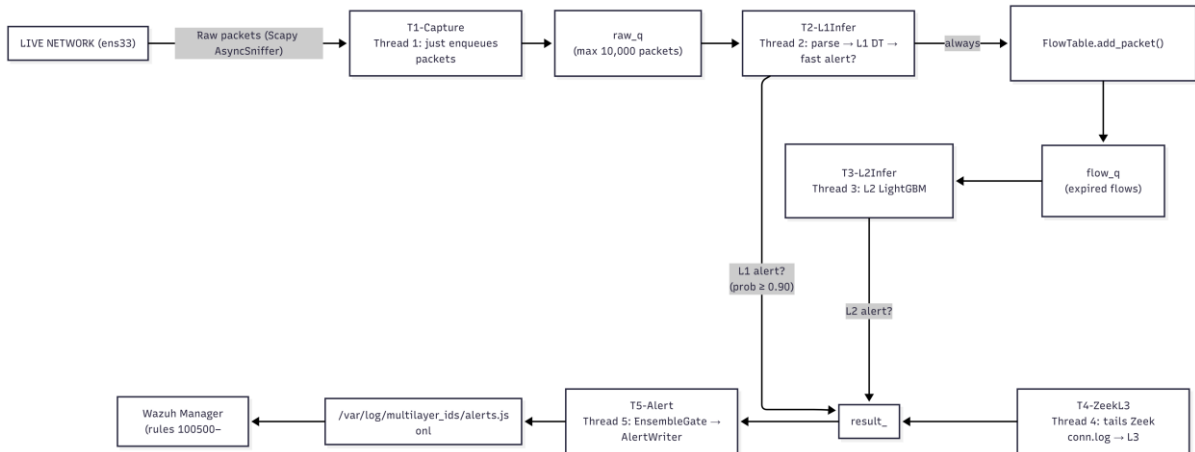
2.1 Mô hình mạng tổng quát

Nhóm triển khai Hệ thống Phát hiện Xâm nhập Mạng Đa lớp (Multi-Layer IDS) bằng AI trong mạng nội bộ 192.168.100.0/24. Hệ thống phân tích lỗi được triển khai trên máy đích là Ubuntu (Ubuntu 20.04 VM) để trực tiếp giám sát, bắt gói tin và xử lý lưu lượng mạng theo thời gian thực. Máy ảo Kali Linux được sử dụng làm máy của kẻ tấn công, thực hiện các kịch bản xâm nhập mạng nhắm vào máy Ubuntu.



Hình 2.1: Mô hình mạng tổng quát

2.2. Thiết kế mô hình phát hiện xâm nhập đa tầng



Hình 2.2: Kiến trúc tổng quát

Mục tiêu kỹ thuật của hệ thống gồm bốn nội dung chính:

- Phân tích được lưu lượng mạng ở nhiều mức biểu diễn thay vì chỉ dựa vào một nguồn tín hiệu.
- Đưa ra cảnh báo sát với thời gian thực, giảm thiểu tối đa độ trễ.
- Chuẩn hóa cảnh báo thành dữ liệu có cấu trúc để Wazuh có thể tiếp nhận và phân tích.
- Cho phép mở rộng thêm mô hình hoặc tầng phát hiện trong tương lai.

Các công nghệ chính được sử dụng gồm:

Scapy để bắt gói tin thời gian thực.

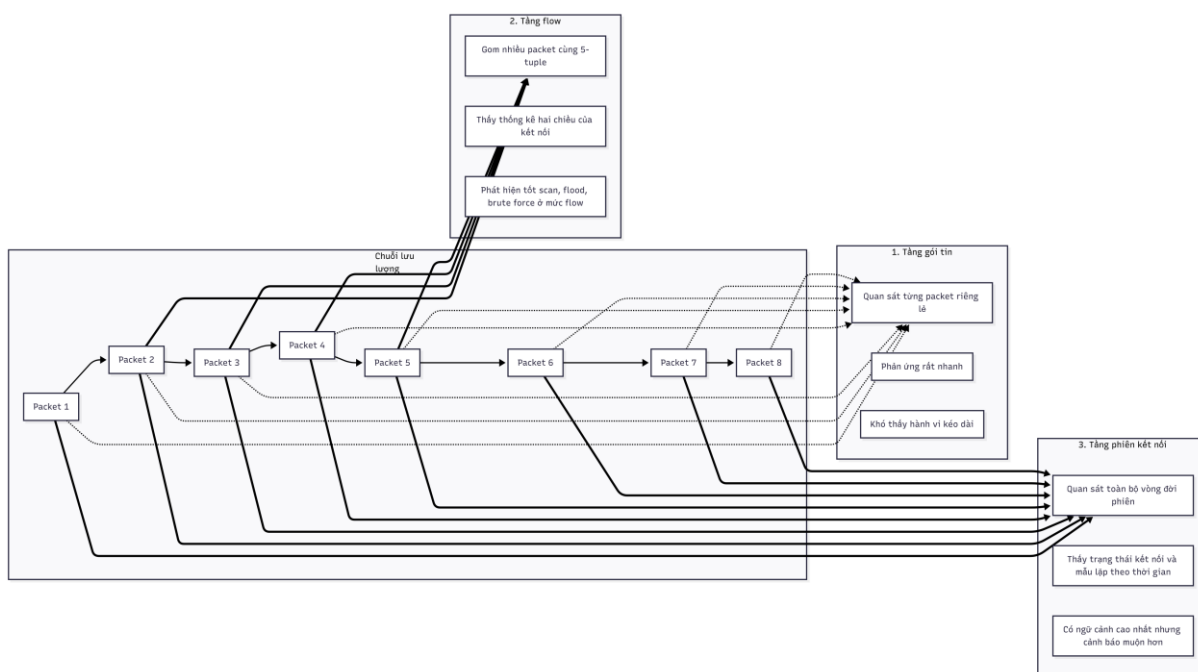
- Zeek để tạo log phiên kết nối, dữ liệu này là đặc trưng của tầng 3.
- Python để xây dựng pipeline xử lý và suy luận.
- Decision Tree: thuật toán cây tìm kiếm cho ra độ chính xác cao nhất, phù hợp với dữ liệu tầng 1.
- LightGBM: thuật toán gọn nhẹ, tốc độ nhanh, phù hợp với dữ liệu cho tầng 2 và tầng 3.
- JSON Lines làm định dạng cảnh báo để Wazuh đọc bằng cơ chế localfile.

2.2.1. Cơ sở lựa chọn kiến trúc đa tầng

Một hệ thống chỉ phân tích ở một mức dữ liệu thường tồn tại vùng mù. Nếu chỉ quan sát ở mức gói tin, hệ thống có thể phản ứng nhanh nhưng khó nắm được hành vi kéo dài trên nhiều kết nối. Nếu chỉ quan sát ở mức flow, hệ thống có thể khai thác được thống kê hai chiều nhưng chưa phản ánh đầy đủ ngữ cảnh phiên. Nếu chỉ quan sát ở mức session, cảnh báo có thể xuất hiện muộn hơn vì phải chờ phiên hình thành hoặc kết thúc. Vì vậy, đề tài lựa chọn kiến trúc đa tầng nhằm kết hợp ưu điểm của cả ba mức quan sát là packet, flow và session. Mô hình được xây dựng với ba tầng phát hiện.

- Tầng 1 quan sát ở mức gói tin và log firewall.
- Tầng 2 quan sát ở mức flow mạng.
- Tầng 3 quan sát ở mức phiên kết nối từ Zeek conn.log.

Việc phân chia như vậy xuất phát từ yêu cầu phải phát hiện được cả tín hiệu bất thường xuất hiện tức thời ở mức packet lẫn các hành vi chỉ bộc lộ rõ khi được quan sát theo flow hoặc theo session.

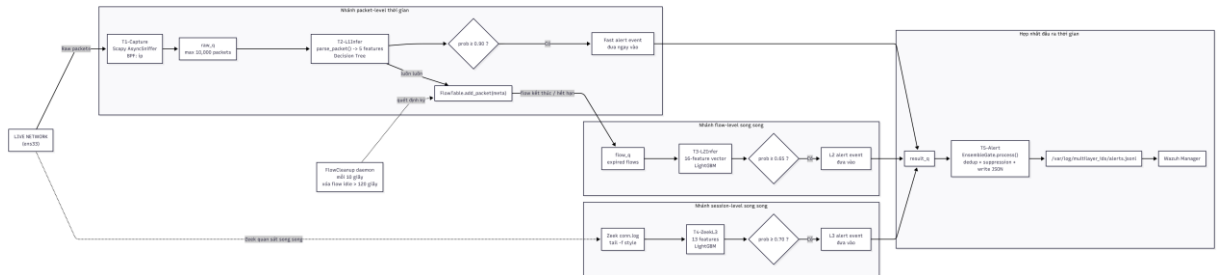


Hình 2.3: Sơ đồ so sánh vùng quan sát của tầng gói tin, tầng flow và tầng phiên kết nối.

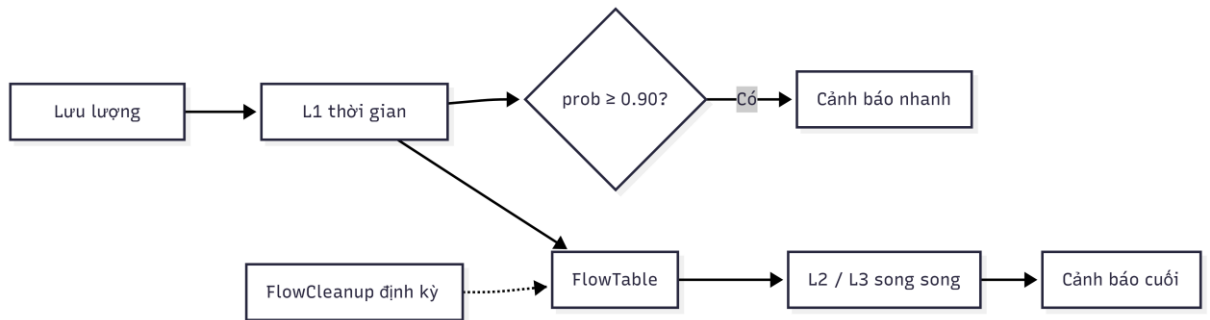
2.2.2. Thiết kế đa luồng đáp ứng cảnh báo thời gian thực

Hệ thống được thiết kế theo mô hình đa luồng để đáp ứng yêu cầu cảnh báo thời gian thực. Thay vì xử lý tuần tự trong một tiến trình duy nhất, các chức năng thu thập gói tin, suy luận ở từng tầng và ghi cảnh báo được tách thành các luồng độc lập, giao tiếp với nhau qua các hàng đợi trung gian. Cách tổ chức này giúp quá trình bắt gói tin không bị chặn bởi thời gian suy luận hay ghi log, đồng thời duy trì khả năng xử lý liên tục khi lưu lượng tăng. Kiến trúc gồm năm luồng chính và một cơ chế dọn dẹp chạy nền:

- T1-Capture dùng Scapy AsyncSniffer để bắt các gói IP trên giao diện ens33 và đưa vào raw_q.
- T2-L1Infer lấy từng gói từ raw_q, trích xuất đặc trưng, suy luận bằng mô hình tầng 1 và sinh cảnh báo nhanh khi xác suất lớn hơn hoặc bằng 0,90. Dù có cảnh báo hay không, gói tin vẫn luôn được cập nhật vào FlowTable để phục vụ tầng 2.
- T3-L2Infer nhận các flow đã kết thúc hoặc hết hạn từ flow_q, xây dựng vector đặc trưng mức flow và suy luận bằng mô hình tầng 2. Nếu xác suất lớn hơn hoặc bằng 0,65, cảnh báo sẽ được đưa vào result_q.
- T4-ZeekL3 hoạt động song song với hai luồng trên, liên tục đọc các dòng mới từ Zeek conn.log, trích xuất đặc trưng mức phiên và suy luận bằng mô hình tầng 3. Nếu xác suất lớn hơn hoặc bằng 0,70, kết quả cũng được đưa vào result_q.
- T5-Alert là luồng hợp nhất cuối cùng. Luồng này lấy các sự kiện từ result_q, gọi EnsembleGate để khử trùng lặp, loại bớt cảnh báo không cần thiết và ghi cảnh báo dưới dạng JSON vào file alerts.jsonl để Wazuh tiếp nhận.
- Bên cạnh năm luồng chính, hệ thống có cơ chế FlowCleanup chạy nền trong FlowTable. Cơ chế này quét bảng trạng thái định kỳ mỗi 10 giây và loại bỏ các flow đã không hoạt động quá 120 giây. Nhờ đó, bộ nhớ được kiểm soát ổn định và hệ thống có thể vận hành liên tục trong thời gian dài.



Hình 2.4: Sơ đồ kiến trúc đa luồng gồm T1-Capture, T2-L1Infer, T3-L2Infer, T4-ZeekL3, T5-Alert và cơ chế FlowCleanup.

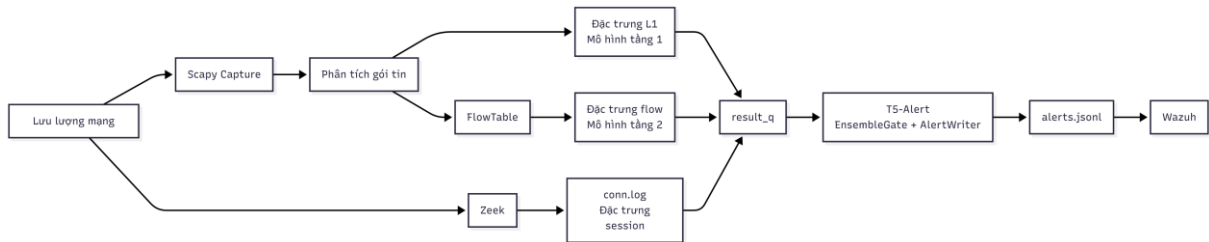


Hình 2.5: Sơ đồ giản lược kiến trúc đa luồng

2.2.3. Luồng dữ liệu thời gian thực

Luồng dữ liệu của hệ thống bắt đầu từ lưu lượng mạng thực trên giao diện ens33. Các gói tin IP được thu thập bởi Scapy AsyncSniffer và đưa vào hàng đợi raw_q. Từ đây, tầng 1 trích xuất đặc trưng mức gói tin để suy luận nhanh bằng mô hình Decision Tree. Đồng thời, thông tin gói tin vẫn luôn được cập nhật vào FlowTable để hình thành flow phục vụ tầng 2. Song song với nhánh này, Zeek theo dõi lưu lượng và ghi dữ liệu phiên vào conn.log, làm đầu vào cho tầng 3. Các kết quả phát hiện từ tầng 1, tầng 2 và tầng 3 được đưa vào result_q, sau đó được T5-Alert xử lý, chuẩn hóa và ghi thành cảnh báo JSON trong file alerts.jsonl. Cuối cùng,

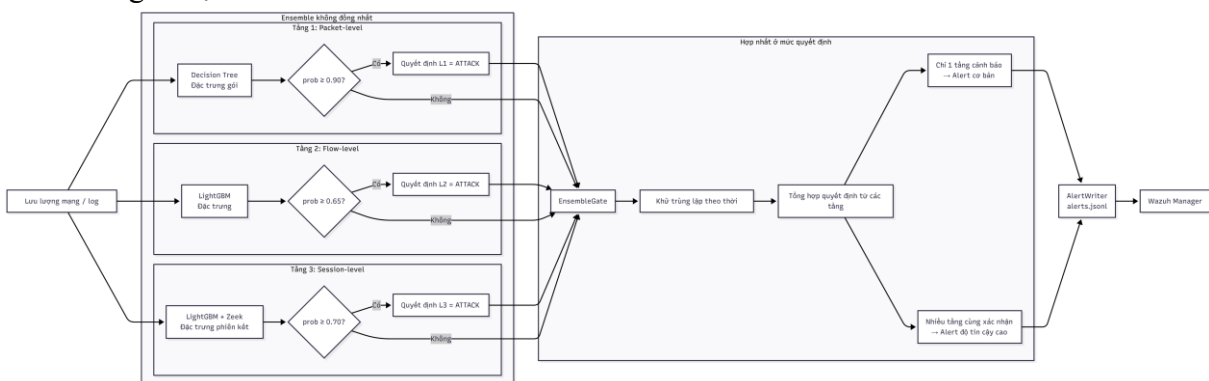
Wazuh đọc file cảnh báo này, áp dụng decoder và rule để hiển thị sự kiện trên hệ thống giám sát.



Hình 2.6: Sơ đồ luồng dữ liệu thời gian thực từ lưu lượng mạng, qua các tầng phát hiện, đến cảnh báo trên Wazuh.

2.2.4. Cơ chế hợp nhất quyết định

Kiến trúc của hệ thống thuộc dạng heterogeneous ensemble, nghĩa là mỗi tầng sử dụng tập đặc trưng và mô hình riêng rồi hợp nhất kết quả ở mức quyết định. Cách thiết kế này phù hợp với bản chất dữ liệu mạng vì mỗi mức quan sát phản ánh một loại tín hiệu khác nhau. Thay vì ép toàn bộ đặc trưng vào cùng một không gian biểu diễn, hệ thống cho phép từng tầng học đúng loại hành vi mà tầng đó quan sát tốt nhất, sau đó mới chuyển kết quả về một điểm hợp nhất chung để tạo cảnh báo.



Hình 2.7: Sơ đồ nguyên lý ensemble không đồng nhất và cơ chế hợp nhất cảnh báo ở mức quyết định.

CHƯƠNG III. CÀI ĐẶT VÀ TRIỂN KHAI

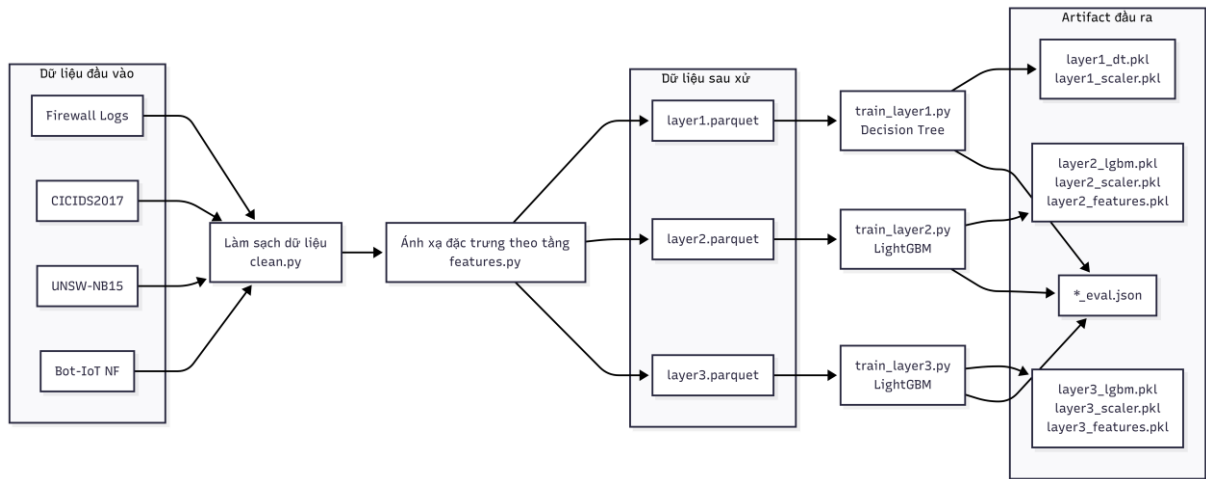
3. Cài đặt và triển khai mô hình phát hiện xâm nhập đa tầng

3.1.1. Môi trường cài đặt

Môi trường kiểm thử của hệ thống gồm một máy Ubuntu chạy IDS engine, Zeek và Wazuh manager; một máy Kali đóng vai trò attacker; và các máy hoặc dịch vụ mục tiêu nằm trong cùng mạng lab. Kiến trúc này cho phép mô phỏng các hành vi tấn công phổ biến, đồng thời quan sát trực tiếp khả năng phát hiện, ghi log và hiển thị cảnh báo của toàn hệ thống trên Wazuh.

3.1.2. Tổng quan quy trình triển khai

Hệ thống được triển khai theo hướng tách phần phát hiện xâm nhập bằng mô hình học máy khỏi phần thu thập và quản lý cảnh báo của Wazuh. Các mô hình Layer 1, Layer 2 và Layer 3 chịu trách nhiệm phân tích lưu lượng mạng ở các mức khác nhau, sau đó sinh cảnh báo dạng JSON. Wazuh đọc các cảnh báo này như một nguồn log nội bộ để phục vụ giám sát, phân loại mức độ nghiêm trọng và phản ứng sự cố.



Hình 3.1: Sơ đồ pipeline từ dữ liệu thô, tiền xử lý, huấn luyện đến lưu artifact triển khai.

3.1.3. Phân tích và xử lý dữ liệu đầu vào

Trong đề tài, toàn bộ dữ liệu được quy về bài toán phân loại nhị phân gồm hai lớp attack và benign. Mục tiêu của cách tiếp cận này là phục vụ yêu cầu xử lý thời gian thực của hệ thống. So với bài toán phân loại đa lớp, bài toán nhị phân cho phép sử dụng mô hình gọn nhẹ hơn, giảm chi phí tính toán, giảm thời gian suy luận và phù hợp hơn với môi trường triển khai có giới hạn tài nguyên. Nhờ đó, hệ thống có thể chạy ổn định trên bộ nhớ RAM khả dụng, rút ngắn độ trễ dự đoán và hỗ trợ sinh cảnh báo sớm ngay khi phát hiện dấu hiệu bất thường trên lưu lượng mạng.

3.1.3.1. Các bộ dữ liệu sử dụng

Dataset	Số dòng	Vai trò	Layer sử dụng
Firewall Logs	65.532	Log firewall mức gói/piên đơn giản	Layer 1
CICIDS2017	2.520.751	Flow mạng, nhiều kiểu tấn công phổ biến	Layer 2
UNSW-NB15	257.673	Flow và session feature	Layer 2, Layer 3
Bot-IoT NF	500.000	NetFlow IoT, botnet, scan, DoS	Layer 2, Layer 3

Bảng 3.1: Các dataset được sử dụng để huấn luyện mô hình

3.1.3.2. Phân tích phân bố nhãn

Layer	Tổng mẫu	Benign	Attack	Nhận xét
Layer 1	65.532	37.640	27.892	Tương đối cân bằng
Layer 2	1.357.199	525.814	831.385	Lệch về attack
Layer 3	853.049	106.803	746.246	Mất cân bằng mạnh

Bảng 3.2: phân bố nhãn

Từ kết quả thống kê sau xử lý có thể thấy phân bố nhãn giữa hai lớp chưa đồng đều ở cả ba tầng, trong đó mức độ mất cân bằng tăng rõ rệt ở Layer 3. Đặc điểm này ảnh hưởng trực tiếp đến cách lựa chọn chỉ số đánh giá cũng như chiến lược huấn luyện mô hình.

3.1.4. Phân tích và xây dựng đặc trưng

Sau bước khảo sát dữ liệu đầu vào, phần tiếp theo tập trung vào việc phân tích đặc trưng và xây dựng bộ biến phù hợp cho từng tầng phát hiện. Mục tiêu của bước này là giữ lại những đặc trưng có giá trị phân biệt cao, đồng thời loại bỏ các biến dư thừa hoặc khó tái tạo trong môi trường triển khai thực tế.

3.1.4.1. Tương quan đặc trưng

Để lựa chọn được các đặc trưng phù hợp, trước hết cần xem xét mức độ tương quan giữa các biến đầu vào và nhãn, cũng như mối liên hệ giữa các biến với nhau. Kết quả phân tích cho thấy mỗi layer có một nhóm đặc trưng nổi bật riêng, phản ánh đúng bản chất dữ liệu ở từng mức quan sát.

Ở Layer 1, dữ liệu chủ yếu phản ánh thông tin ở mức packet hoặc firewall log, do đó số lượng đặc trưng không nhiều nhưng vẫn thể hiện tín hiệu phân tách tương đối rõ.

- nat_src_port và dst_port có tương quan rõ với nhãn.
- total_bytes, pkt_count, pkts_sent liên quan mạnh với nhau.
- Dữ liệu ít feature, phù hợp mô hình nhẹ.

Đối với Layer 2, dữ liệu flow mang tính tổng hợp cao hơn nên hành vi tấn công không thể được biểu diễn chỉ bởi một biến đơn lẻ. Thay vào đó, mô hình cần học trên sự kết hợp của nhiều đặc trưng về byte, packet, thời lượng và cờ TCP.

- Các feature quan trọng: fwd_bytes, flow_dur_us, flow_bytes_s, pkt_len_min, bwd_bytes, dst_port.
- Hành vi tấn công flow không nằm ở một feature duy nhất mà là tổ hợp byte, packet, duration, TCP flag và protocol.

Ở Layer 3, dữ liệu session cung cấp thêm ngữ cảnh kết nối và các thống kê theo phiên, nhờ đó giúp mô hình nhận diện tốt hơn những hành vi kéo dài hoặc lặp lại theo thời gian. Tuy nhiên, đây cũng là tầng dễ bị ảnh hưởng bởi chênh lệch giữa dữ liệu huấn luyện và log Zeek khi chạy thực tế.

- Feature quan trọng: fwd_bytes, bwd_bytes, flow_dur_us, ct_srv_dst, fwd_pkts, sload, src_ttl, state_enc.
- Có tín hiệu session nhưng dễ bị ảnh hưởng bởi khác biệt giữa dataset và môi trường Zeek thật.

3.1.4.2. Chuẩn hóa nhãn

Do các bộ dữ liệu đầu vào được xây dựng từ nhiều nguồn khác nhau và sử dụng hệ thống nhãn khác nhau, cần có một bước chuẩn hóa nhãn thống nhất trước khi huấn luyện. Trong đề tài này, toàn bộ dữ liệu được quy về bài toán phân loại nhị phân, trong đó 0 biểu diễn lưu lượng bình

thường và 1 biểu diễn lưu lượng tấn công hoặc bất thường. Cách chuẩn hóa này giúp đơn giản hóa bài toán, đồng thời phù hợp với yêu cầu phát hiện thời gian thực của hệ thống.

Cách chuyển:

- Firewall Logs: allow = 0, còn deny/drop/reset-both = 1.
- CICIDS2017: Normal Traffic = 0, các lớp còn lại = 1.
- UNSW-NB15: dùng trực tiếp label.
- Bot-IoT NF: dùng trực tiếp Label.

3.1.4.3. Làm sạch dữ liệu

Sau khi thống nhất nhãn, dữ liệu tiếp tục được làm sạch để loại bỏ các thành phần gây nhiễu, giảm nguy cơ học sai lệch và bảo đảm các đặc trưng đầu vào có thể sử dụng được trong môi trường triển khai thật. Đây là bước quan trọng nhằm tăng độ ổn định của mô hình và giữ cho pipeline xử lý nhất quán giữa giai đoạn huấn luyện và giai đoạn suy luận.

Các bước thực hiện:

- Xóa khoảng trắng tên cột.
- Loại bỏ giá trị NaN, inf, -inf.
- Mã hóa cột categorical như proto, state, service.
- Xóa cột định danh hoặc timestamp không dùng được khi triển khai thật.
- Xóa cột rò rỉ nhãn như attack_cat, Attack.
- Loại bỏ các cột dư thừa có tương quan quá cao.
- Clip outlier ở mức phân vị 99.9% cho một số feature số.

3.1.4.4. Xây dựng các đặc trưng cho từng layer

Trên cơ sở kết quả phân tích tương quan và làm sạch dữ liệu, bộ đặc trưng đầu vào được xây dựng riêng cho từng layer. Cách thiết kế này giúp mỗi mô hình chỉ sử dụng những biến phù hợp nhất với mức quan sát mà nó đảm nhiệm, tránh làm tăng độ phức tạp không cần thiết.

Với Layer 1, mục tiêu là phát hiện nhanh các tín hiệu bất thường ở mức packet hoặc firewall log. Vì vậy, bộ đặc trưng được giữ ở mức gọn nhẹ để giảm độ trễ suy luận.

- dst_port, nat_src_port, total_bytes, pkt_count, pkts_sent

Layer 2 làm việc trên dữ liệu flow, do đó bộ đặc trưng được mở rộng để phản ánh đầy đủ hơn đặc điểm lưu lượng hai chiều, thời lượng kết nối, tốc độ truyền và hành vi TCP.

- dst_port, flow_dur_us, fwd_pkts, fwd_bytes,
- bwd_pkts, bwd_bytes, pkt_len_min, pkt_len_avg,
- flow_bytes_s, psh_flag, ack_flag, fin_flag,
- init_win_fwd, active_mean, protocol_enc, src

Ở Layer 3, các đặc trưng được lựa chọn theo hướng nhấn mạnh ngữ cảnh phiên, trạng thái kết nối và các thống kê có khả năng phản ánh hành vi lặp lại hoặc bất thường kéo dài.

- flow_dur_us, fwd_bytes, bwd_bytes, fwd_pkts,
- bwd_pkts, src_ttl, dst_ttl, sload, dload,
- state_enc, ct_dst_sport_ltm, ct_srv_dst, src

Bên cạnh các đặc trưng kỹ thuật, biến src được bổ sung để biểu diễn nguồn dataset. Việc đưa biến này vào mô hình là cần thiết vì các bộ dữ liệu sử dụng trong đề tài có schema khác nhau và mức độ thiếu đặc trưng không giống nhau. Nhờ đó, mô hình có thể học được khác biệt giữa các nguồn dữ liệu, tránh hiểu nhầm giá trị 0 do thiếu feature là giá trị thật của lưu lượng mạng. Trường src biểu diễn nguồn dataset. Do các bộ dữ liệu có schema và đặc trưng thiếu khác nhau, biến này giúp mô hình học được sự khác biệt giữa các nguồn dữ liệu, tránh hiểu nhầm giá trị 0 do thiếu feature là giá trị thật của lưu lượng mạng.

3.1.5. Lựa chọn thuật toán

Sau khi xác định được bộ đặc trưng cho từng layer, bước tiếp theo là lựa chọn thuật toán huấn luyện phù hợp. Việc lựa chọn này không chỉ dựa trên độ chính xác, mà còn phải xét đến khả năng triển khai thực tế, tốc độ suy luận và mức độ phù hợp với dữ liệu mạng dạng bảng.

Các tiêu chí cân nhắc như sau:

- Phù hợp dữ liệu dạng bảng, ma trận trọng số.
- Suy luận nhanh trong môi trường thời gian thực.
- Xử lý được quan hệ phi tuyến.
- Chịu được dữ liệu mất cân bằng.
- Có thể xuất mô hình và nạp lại khi triển khai.
- Không yêu cầu GPU.

Với Layer 1, mục tiêu chính là phát hiện nhanh các tín hiệu bất thường ở mức packet/firewall. Do số lượng đặc trưng nhỏ và yêu cầu độ trễ thấp, Decision Tree được chọn thay vì các mô hình phức tạp hơn. Lựa chọn này giúp hệ thống đạt tốc độ suy luận nhanh, dễ giải thích và phù hợp với tầng cảnh báo sớm.

Layer 2 và Layer 3 sử dụng LightGBM vì dữ liệu flow và session có nhiều đặc trưng số, phân bố lệch và quan hệ phi tuyến. LightGBM cho phép học ranh giới phức tạp giữa benign và attack nhưng vẫn giữ được tốc độ suy luận phù hợp triển khai thời gian thực. Đồng thời, mô hình này cũng thuận lợi trong việc lưu trữ, nạp lại và tích hợp trực tiếp vào pipeline vận hành của hệ thống.

3.1.6. Huấn luyện mô hình

Quy trình huấn luyện mô hình được trình bày tổng quát theo bảng dưới đây:

STT	Nội dung
1	Đọc dữ liệu đã xử lý từ file Parquet
2	Tách feature X và nhãn y
3	Chia dữ liệu train/test theo tỷ lệ 70/30
4	Dùng stratified split để giữ phân bố nhãn
5	Chuẩn hóa feature bằng StandardScaler
6	Kiểm tra mất cân bằng lớp
7	Áp dụng SMOTE nếu tỷ lệ lệch lớp lớn hơn 5:1
8	Huấn luyện mô hình
9	Đánh giá bằng Accuracy, Precision, Recall, F1, AUC
10	Lưu model, scaler và feature list

Bảng 3.3: Khái quát quy trình huấn luyện mô hình

3.1.7. Thông số huấn luyện các mô hình

Layer 1 là tầng phát hiện nhanh ở mức packet hoặc log firewall, do đó yêu cầu quan trọng nhất là tốc độ suy luận thấp và mô hình đủ gọn để xử lý tức thời. Với số lượng đặc trưng ít và khả năng phân tách tương đối rõ của dữ liệu firewall, mô hình Decision Tree được lựa chọn cho tầng này. Cấu hình huấn luyện được thiết lập theo hướng cân bằng giữa khả năng tổng quát hóa và tốc độ thực thi, trong đó class_weight được đặt ở chế độ balanced để hạn chế ảnh hưởng của mất cân bằng lớp.

Thành phần	Giá trị
Model	DecisionTreeClassifier

Train/test	70/30
max_depth	15
min_samples_leaf	5
class_weight	balanced
Scaler	StandardScaler
Output	layer1_dt.pkl, layer1_scaler.pkl

Bảng 3.4: Thông số mô hình tầng 1

Layer 2 là tầng phát hiện chính ở mức flow, nơi dữ liệu có số lượng mẫu lớn, đặc trưng đa dạng và quan hệ phi tuyến tương đối rõ. Vì vậy, LightGBM được sử dụng để khai thác tốt các đặc trưng dạng bảng, đồng thời vẫn bảo đảm tốc độ huấn luyện và suy luận phù hợp cho môi trường triển khai thời gian thực. Bộ tham số được lựa chọn nhằm duy trì hiệu quả phát hiện cao nhưng không làm mô hình quá phức tạp.

Thành phần	Giá trị
Model	LGBMClassifier
Train/test	70/30
n_estimators	200
max_depth	8
num_leaves	63
learning_rate	0.1
subsample	0.8
colsample_bytree	0.8
class_weight	balanced
Scaler	StandardScaler
Output	layer2_lgbm.pkl, layer2_scaler.pkl, layer2_features.pkl

Bảng 3.5: Thông số mô hình tầng 2

Layer 3 xử lý dữ liệu ở mức session, do đó mô hình cần phản ánh được ngữ cảnh kết nối và các dấu hiệu bất thường xuất hiện trong suốt vòng đời phiên mạng. Tương tự Layer 2, LightGBM tiếp tục được sử dụng vì phù hợp với dữ liệu bảng và có khả năng học tốt các quan hệ phi tuyến giữa các đặc trưng session như thời lượng kết nối, tải, TTL và trạng thái phiên. So với Layer 2, cấu hình của Layer 3 được điều chỉnh theo hướng gọn hơn để phù hợp với tập đặc trưng và độ phức tạp thực tế của dữ liệu tầng này.

Thành phần	Giá trị
Model	LGBMClassifier
Train/test	70/30
n_estimators	150
max_depth	7
num_leaves	31
learning_rate	0.1
subsample	0.8
colsample_bytree	0.8
class_weight	balanced
Scaler	StandardScaler
Output	layer3_lgbm.pkl, layer3_scaler.pkl, layer3_features.pkl

Bảng 3.6: Thông số mô hình tầng 3

3.1.8. Kết quả huấn luyện

Sau quá trình huấn luyện, cả ba tầng đều đạt kết quả tốt trên tập kiểm thử. Các chỉ số Accuracy, Precision, Recall, F1 và AUC cho thấy mô hình có khả năng phân biệt hai lớp benign và attack ở mức cao. Trong đó, Layer 1 đạt kết quả nổi bật nhờ không gian đặc trưng nhỏ và dữ liệu firewall có tính phân tách tốt; Layer 2 thể hiện vai trò là tầng phát hiện chính với hiệu quả ổn định trên dữ liệu flow lớn; còn Layer 3 duy trì Precision rất cao nhưng Recall thấp hơn do dữ liệu session trong môi trường thực tế có thể khác biệt nhất định so với dữ liệu huấn luyện.

Layer	Model	Test samples	Accuracy	Precision	Recall	F1	AUC
Layer 1	DT	19.660	0.9983	0.9975	0.9984	0.9980	0.9992
Layer 2	LightGBM	407.160	0.9822	0.9861	0.9848	0.9854	0.9989
Layer 3	LightGBM	255.915	0.9535	0.9940	0.9526	0.9729	0.9942

Bảng 3.7: Các chỉ số đo lường hiệu quả

Ma trận nhầm lẫn của từng tầng tiếp tục cho thấy khả năng phát hiện tổng thể của hệ thống là tích cực. Số lượng false positive và false negative của Layer 1 và Layer 2 ở mức thấp, trong khi Layer 3 có số lượng false negative cao hơn tương đối. Điều này phản ánh đặc trưng session chịu ảnh hưởng nhiều hơn từ sự khác biệt giữa dữ liệu huấn luyện và dữ liệu sinh ra trong môi trường triển khai thật.

Layer	TN	FP	FN	TP
Layer 1	11.271	21	13	8.355
Layer 2	154.275	3.469	3.788	245.628
Layer 3	30.747	1.294	10.606	213.268

Bảng 3.8: Ma trận rời

Từ các kết quả trên có thể rút ra một số nhận xét chính. Thứ nhất, Layer 1 đạt kết quả rất cao do các đặc trưng firewall có khả năng phân tách rõ giữa lưu lượng bình thường và bất thường. Thứ hai, Layer 2 là tầng phát hiện trọng tâm, đạt F1 và AUC cao trên tập dữ liệu flow có quy mô lớn, cho thấy mô hình phù hợp với nhiệm vụ phát hiện tấn công ở mức lưu lượng tổng hợp. Thứ ba, Layer 3 có Precision cao nhưng Recall thấp hơn do một số đặc trưng session, đặc biệt là TTL và trạng thái kết nối, có thể thay đổi đáng kể giữa bộ dữ liệu huấn luyện và môi trường triển khai thật. Cuối cùng, AUC của cả ba tầng đều ở mức rất cao, cho thấy hệ thống có năng lực phân biệt benign và attack tốt trên toàn bộ pipeline huấn luyện.

3.1.9. Ngưỡng cảnh báo

Do đặc điểm dữ liệu và vai trò của từng tầng khác nhau, hệ thống không sử dụng chung một ngưỡng cảnh báo cho toàn bộ pipeline. Thay vào đó, mỗi layer được gán một ngưỡng riêng để cân bằng giữa khả năng phát hiện và mức độ cảnh báo nhầm. Layer 1 chỉ quan sát ít đặc trưng nên được đặt ngưỡng cao để bảo đảm chỉ phát cảnh báo nhanh khi mô hình thực sự tự tin. Layer 2 là tầng phát hiện chính nên chọn ngưỡng thấp hơn để duy trì khả năng phát hiện tốt trên flow. Layer 3 sử dụng ngưỡng trung gian cao hơn Layer 2 nhằm giảm nhiễu do sự khác biệt giữa dữ liệu session huấn luyện và log thật trong quá trình triển khai.

Layer	Ngưỡng	Vai trò
Layer 1	0.90	Fast-path, chỉ cảnh báo khi rất tự tin
Layer 2	0.65	Tầng phát hiện flow chính

Layer 3	0.70	Bổ sung cảnh báo session, giảm nhiễu
---------	------	--------------------------------------

Bảng 3.9: Ngưỡng cảnh báo cho từng tầng

3.1.10. Cơ chế chống trùng lặp

Một thách thức quan trọng trong hệ thống phát hiện xâm nhập thời gian thực là cùng một đợt tấn công có thể sinh ra rất nhiều packet, flow hoặc session gần giống nhau. Nếu mọi sự kiện đều được ghi thành cảnh báo độc lập, Wazuh sẽ nhận quá nhiều alert lặp và gây nhiễu cho quá trình theo dõi, điều tra cũng như phân tích chuỗi tấn công. Vì vậy, hệ thống sử dụng cơ chế khử trùng lặp riêng cho từng layer dựa trên khóa nhận diện sự kiện và một cửa sổ thời gian xác định.

Layer	Quy tắc dedup	Thời gian
Layer 1	(src_ip, dst_ip, dst_port, L1)	15 giây
Layer 2	(src_ip, dst_ip, dst_port, L2)	30 giây
Layer 3	(src_ip, dst_ip, L3)	600 giây

Bảng 3.10: Khoảng thời gian áp dụng khử trùng lặp

3.1.11. Định dạng cảnh báo đầu ra

Đầu ra của hệ thống được chuẩn hóa theo định dạng JSON Lines, trong đó mỗi dòng tương ứng với một cảnh báo độc lập. Cách tổ chức này giúp pipeline ghi log đơn giản, dễ mở rộng trường dữ liệu và đặc biệt phù hợp với cơ chế đọc file theo thời gian thực của Wazuh. Mỗi cảnh báo chứa các thông tin tối thiểu để phục vụ phát hiện, đối chiếu và điều tra, bao gồm thời gian phát hiện, kết luận của mô hình, độ tin cậy, tầng phát hiện và các thông tin mạng liên quan như địa chỉ IP nguồn, IP đích và cổng đích.

Ví dụ cấu trúc cảnh báo đầu ra như sau:

```
{
  "timestamp": "2026-04-11T12:00:00Z",
  "verdict": "ATTACK",
  "confidence": 0.92,
  "layers": ["L2"],
  "src_ip": "192.168.100.101",
  "dst_ip": "192.168.100.103",
  "dst_port": 22,
  "proto": "6",
  "flow_dur_us": 5000.0
}
```

Trong cấu trúc trên, trường confidence biểu diễn mức độ tin cậy của mô hình đối với dự đoán hiện tại. Trường layers cho biết cảnh báo được sinh ra từ tầng nào, ví dụ L1, L2 hoặc L3. Các trường src_ip, dst_ip và dst_port có vai trò quan trọng trong quá trình điều tra trên Wazuh, vì chúng cho phép đối chiếu với log hệ thống, log agent và các sự kiện mạng liên quan. Việc sử dụng JSON Lines giúp Wazuh có thể đọc từng cảnh báo ngay khi file được cập nhật, từ đó hỗ trợ giám sát và cảnh báo gần thời gian thực.

3.1.12. Triển khai IDS engine thời gian thực

IDS engine được xây dựng theo mô hình đa luồng để tách biệt các bước thu thập, xử lý, suy luận và ghi cảnh báo. Thiết kế này giúp hệ thống hoạt động ổn định hơn trong điều kiện lưu lượng thay đổi, đồng thời tránh việc một khâu xử lý chậm làm nghẽn toàn bộ pipeline. Mỗi thành phần đảm nhiệm một nhiệm vụ riêng và giao tiếp với nhau thông qua queue.

Thread	Thành phần	Vai trò
--------	------------	---------

T1	Capture	Bắt packet bằng Scapy
T2	L1 Inference	Suy luận packet-level
T3	L2 Inference	Suy luận flow-level
T4	Zeek L3 Reader	Đọc conn.log, suy luận session-level
T5	Alert Writer	Ghi alert JSON Lines

Bảng 3.11: Các luồng chạy song song

Để tránh tình trạng tràn bộ nhớ khi lưu lượng tăng cao, các queue được cấu hình với kích thước giới hạn. Khi số lượng packet hoặc flow vượt quá ngưỡng cho phép, hệ thống ưu tiên giữ tính ổn định chung thay vì để hàng đợi tăng vô hạn và làm cạn tài nguyên RAM. Đây là một yêu cầu quan trọng đối với môi trường triển khai thời gian thực.

Các model và scaler sau huấn luyện được lưu trong thư mục models/ và được nạp lại khi khởi động engine. Việc tách riêng bước huấn luyện và bước suy luận giúp hệ thống triển khai gọn hơn, đồng thời bảo đảm mô hình chạy thực tế đúng với cấu hình đã được kiểm thử trước đó. Toàn bộ cảnh báo sau khi qua cơ chế kiểm tra ngưỡng và khử trùng lặp sẽ được ghi vào file /var/log/multilayer_ids/alerts.jsonl để Wazuh thu thập.

CHƯƠNG IV. KIỂM THỬ VÀ ĐÁNH GIÁ

4.1. Mục tiêu kiểm thử

Chương này trình bày quá trình kiểm thử và đánh giá hệ thống phát hiện xâm nhập đa tầng sau khi đã hoàn thành huấn luyện mô hình và tích hợp với Wazuh.

Các mục tiêu kiểm thử gồm:

1. Đánh giá độ chính xác của từng mô hình Layer 1, Layer 2 và Layer 3 trên tập test.
2. Kiểm thử khả năng phát hiện trong môi trường triển khai thật.
3. Đánh giá độ trễ xử lý cảnh báo từ lúc mô hình suy luận đến khi ghi log.
4. Đánh giá tài nguyên hệ thống khi chạy IDS engine, gồm CPU và RAM.
5. Kiểm tra việc Wazuh đọc, giải mã và phân loại cảnh báo IDS.
6. Phân tích các hạn chế còn tồn tại khi triển khai mô hình trong môi trường thực tế.

4.2. Môi trường kiểm thử

Hệ thống được kiểm thử trong môi trường lab gồm các máy ảo cùng mạng nội bộ.

Thành phần	Vai trò
Ubuntu VM	Chạy IDS engine, Zeek và Wazuh manager
Kali Linux VM	Máy tấn công, dùng để sinh lưu lượng kiểm thử
Zeek	Sinh log phiên kết nối conn.log cho Layer 3
IDS Engine	Chạy mô hình Layer 1, Layer 2, Layer 3 và ghi alert JSON

Bảng 4.1: Môi trường

IDS engine ghi cảnh báo vào: `/var/log/multilayer_ids/alerts.jsonl`

4.3. Phương pháp kiểm thử

Quá trình kiểm thử được chia thành hai nhóm chính:

Nhóm kiểm thử	Mục đích
Kiểm thử offline	Đánh giá mô hình trên tập test 30% chưa dùng khi huấn luyện
Kiểm thử live	Chạy IDS engine trong môi trường thật và sinh tấn công từ Kali

Bảng 4.2: Các nhóm kiểm thử

Kiểm thử offline giúp đánh giá năng lực phân loại của mô hình trên dữ liệu đã chuẩn hóa. Kiểm thử live giúp đánh giá khả năng triển khai thực tế, bao gồm độ trễ, tài nguyên sử dụng và khả năng tích hợp Wazuh.

4.4. Đánh giá offline trên tập test

4.4.1. Metric sử dụng

Các metric được sử dụng:

Metric	Ý nghĩa
Accuracy	Tỷ lệ dự đoán đúng tổng thể
Precision	Trong các cảnh báo attack, có bao nhiêu cảnh báo đúng
Recall	Trong các mẫu attack thật, mô hình phát hiện được bao nhiêu
F1-score	Trung bình điều hòa giữa Precision và Recall
AUC	Khả năng phân tách giữa benign và attack
Confusion Matrix	Phân tích TN, FP, FN, TP

Bảng 4.3: Metric

Do dữ liệu có mất cân bằng, đặc biệt ở Layer 3, báo cáo không chỉ dựa vào Accuracy mà cần xem thêm Precision, Recall, F1 và AUC.

4.4.2. Kết quả offline

Layer	Model	Mẫu	Accuracy	Precision	Recall	F1	AUC
Layer 1	Decision Tree	19.660	0.9983	0.9975	0.9984	0.9980	0.9992
Layer 2	LightGBM	407.160	0.9822	0.9861	0.9848	0.9854	0.9989
Layer 3	LightGBM	255.915	0.9535	0.9940	0.9526	0.9729	0.9942

Bảng 4.4: Kết quả offline

Nhận xét:

- Layer 1 đạt kết quả cao nhất do dữ liệu firewall có đặc trưng phân tách rõ.
- Layer 2 đạt F1 = 0.9854, cho thấy đây là tầng phát hiện chính ở mức flow.
- Layer 3 có Precision cao nhưng Recall thấp hơn, nghĩa là khi Layer 3 cảnh báo thì thường đúng, nhưng vẫn có một số attack bị bỏ sót.
- AUC của cả ba layer đều trên 0.99 hoặc xấp xỉ 0.99, cho thấy các mô hình có khả năng phân biệt benign/attack tốt.

4.4.3. Confusion matrix

```

### Confusion matrices (TN / FP / FN / TP)

- **L1**: TN=11,271  FP=21  FN=13  TP=8,355
- **L2**: TN=154,277  FP=3,467  FN=3,799  TP=245,617
- **L3**: TN=30,746  FP=1,295  FN=10,605  TP=213,269

- - -

```

Hình 4.1: Bảng Confusion matrix

Nhận xét:

- Layer 1 có số FP và FN rất thấp.
- Layer 2 có số mẫu test lớn nhất nên số lượng lỗi tuyệt đối cao hơn, nhưng tỷ lệ lỗi vẫn thấp.
- Layer 3 có FN cao hơn, phản ánh việc một số đặc trưng session trong dataset không chuyển giao hoàn toàn sang môi trường triển khai thật.

4.5. Kiểm thử live trong môi trường triển khai

4.5.1. Kịch bản kiểm thử

Các kịch bản tấn công được thực hiện từ Kali Linux:

Kịch bản	Mục tiêu kiểm thử	Layer kỳ vọng
SYN scan bằng Nmap	Kiểm tra phát hiện reconnaissance	Layer 2
UDP scan	Kiểm tra scan nhiều cổng/giao thức	Layer 2
Port sweep	Kiểm tra nhiều kết nối ngán	Layer 2, Layer 3
SYN flood	Kiểm tra lưu lượng bất thường mức packet/flow	Layer 1, Layer 2
SSH brute force	Kiểm tra hành vi lặp đến dịch vụ SSH	Layer 2, Layer 3

Bảng 4.5: Kịch bản kiểm thử

4.5.2. Kết quả cảnh báo live

Trong lượt chạy kiểm thử live, IDS engine sinh cảnh báo từ các layer như sau:

```
## 2. Live Detection Summary

- **Total alerts fired**: 37
- **L1 alerts**: 2
- **L2 alerts**: 35
- **L3 alerts**: 0
```

Hình 4.2: Kết quả Live Detection

Tổng số cảnh báo ghi nhận trong phase kiểm thử là 37 cảnh báo. Trong đó, các cảnh báo thuộc Layer 1, Layer 2 và Layer 3 là các cảnh báo phục vụ đánh

giá mô hình IDS ba tầng.

Nhận xét:

- Layer 2 là tầng sinh nhiều cảnh báo nhất, phù hợp với vai trò tầng phát hiện chính ở mức flow.

- Layer 1 chỉ cảnh báo khi confidence cao, do đó số lượng ít nhưng có ý nghĩa cảnh báo nhanh.

- Layer 3 không sinh cảnh báo hơn do ngưỡng được đặt cao hơn và do đặc trưng session phụ thuộc vào log Zeek, vẫn đề thiếu đặc trưng TTL từ log của Zeek.

4.6. Đánh giá độ trễ xử lý

```
## 3. End-to-End Alert Latency (inference → alert written)

_Measures time from inference completion to JSON line written._
```

Layer	Mean (ms)	p50 (ms)	p95 (ms)	Max (ms)
L1	3.643	4.777	4.777	4.777
L2	2.853	0.653	19.925	23.661
L3	-	-	-	-

Hình 4.3: Độ trễ xử lý

Nhận xét:

- Độ trễ ghi alert của cả ba layer đều ở mức mili-giây.

- Layer 2 có độ trễ trung bình cao hơn Layer 1 và Layer 3 trong lượt đo này, nhưng vẫn rất thấp.

- Layer 3 do không sinh cảnh báo nên không đo được độ trễ.

- Kết quả cho thấy việc ghi cảnh báo JSON Lines không tạo ra nút nghẽn đáng kể.

4.7. Đánh giá throughput

Trong phase kiểm thử, hệ thống ghi nhận:

4. Throughput

Metric	Value
Packets processed	31,593
Flows processed	778
Packet throughput	149.6 pkt/s
Flow throughput	3.7 flows/s
Capture duration	211.2s

Hình 4.4: Throughput

Nhận xét:

Thông lượng thực tế ghi nhận ở mức thấp: Trong thời gian đo đặc là 211.2 giây, hệ thống xử lý tổng cộng 31.593 gói tin và 778 luồng, tương đương với tốc độ chỉ đạt 149.6 gói/giây và 3.7 luồng/giây. Nguyên nhân là do môi trường lab kiểm thử chỉ có các luồng tấn công từ máy Kali mà không có lượng lớn lưu lượng mạng nền như môi trường doanh nghiệp thực tế.

Mục tiêu là kiểm chứng luồng xử lý: Trọng tâm của giai đoạn này là xác nhận sự vận hành ổn định của kiến trúc AI đa luồng, chứ không phải để ép tải đo thông lượng tối đa của hệ thống.

Định hướng kiểm thử tải cao :Để đánh giá được giới hạn chịu đựng cực hạn của cỗ máy IDS này (vốn có năng lực lý thuyết lên tới hàng triệu gói tin/giây), các kịch bản kiểm thử sau này sẽ cần sử dụng công cụ sinh lưu lượng mạng liên tục và dồn dập hơn để ép cỗ máy AI chạy hết công suất.

4.8. Đánh giá tài nguyên CPU và RAM

Tài nguyên sử dụng của tiến trình IDS engine được đo trong quá trình chạy phase kiểm thử.

5. Resource Usage (IDS engine process)

Metric	Mean	Peak
CPU	17.3%	90.9%
RAM	282.9 MB	312.6 MB

Hình 4.5: Tài nguyên sử dụng

CPU trung bình thấp, cho thấy hệ thống gây tải không quá lớn trong môi trường kiểm thử.

CPU peak 90.9% xuất hiện khi có xử lý packet/flow hoặc ghi cảnh báo.

RAM trung bình khoảng 282.9 MB, peak khoảng 312.6 MB, phù hợp với môi trường máy ảo.

Việc nạp đồng thời ba mô hình, scaler, queue và flow table không tạo áp lực bộ nhớ lớn.

Kết quả đo tài nguyên cho thấy IDS engine có khả năng chạy ổn định trên máy ảo CPU-only. Mức RAM cao nhất dưới 313 MB và CPU trung bình dưới 18%, chứng tỏ mô hình và pipeline xử lý phù hợp với mục tiêu triển khai nhẹ, không yêu cầu GPU hoặc tài nguyên máy chủ lớn.

4.9. Phân tích kết quả theo từng layer

4.9.1. Layer 1

Layer 1 hoạt động tốt với các tín hiệu đơn giản, rõ ở mức packet/firewall. Do sử dụng Decision Tree và số lượng feature ít, tầng này có tốc độ rất cao.

Ưu điểm:

- Độ trễ thấp.
- Dễ giải thích.
- Phù hợp cảnh báo nhanh.

Hạn chế:

- Không đủ ngữ cảnh để phát hiện các hành vi phức tạp. - Cần kết hợp với Layer 2 và Layer 3 để giảm bỏ sót.

4.9.2. Layer 2

Layer 2 là tầng phát hiện chính. Kết quả offline và live đều cho thấy phần lớn cảnh báo đến từ tầng này.

Ưu điểm:

- Học được hành vi flow.
- Phát hiện tốt scan, brute force, DoS/DDoS.
- Cân bằng tốt giữa Precision và Recall.

Hạn chế:

- Phụ thuộc vào chất lượng gom flow.
- Một số flow ngắn hoặc thiếu chiều phản hồi có thể làm feature bị lệch.

4.9.3. Layer 3

Layer 3 bổ sung góc nhìn session dựa trên Zeek conn.log.

Ưu điểm:

- Có thêm thông tin trạng thái kết nối và thống kê phiên.
- Precision cao, cảnh báo thường có độ tin cậy tốt.

Hạn chế:

- Recall thấp hơn Layer 1 và Layer 2.

- Trong môi trường chạy thực tế (live), Zeek không ghi nhận các thông số thời gian sống của gói tin (src_ttl và dst_ttl), làm suy giảm sức mạnh dự báo của mô hình.

- Cần hiệu chỉnh thêm theo mạng triển khai thực tế.

4.10. Hạn chế và hướng cải thiện

Các hạn chế hiện tại:

1. Môi trường live test còn nhỏ, lưu lượng nền thấp.
2. Throughput chưa phản ánh giới hạn tối đa của hệ thống.
3. Layer 3 phụ thuộc vào đặc trưng session có thể khác giữa dataset và log thật.
4. Một số cảnh báo có thể bị ảnh hưởng bởi traffic nội bộ như DNS, DHCP, NTP hoặc phản hồi từ nạn nhân.
5. Mô hình hiện phân loại nhị phân, chưa phân loại chi tiết từng loại tấn công.

Hướng cải thiện:

1. Kiểm thử với lưu lượng lớn hơn để đo throughput tối đa.
2. Hiệu chỉnh threshold theo từng môi trường mạng.
3. Giảm phụ thuộc của Layer 3 vào TTL, bổ sung feature hành vi phiên ổn định hơn.
4. Kết hợp alert IDS với log agent trên Wazuh để đánh giá theo chuỗi tấn công.
5. Mở rộng đánh giá theo từng nhóm tấn công: scan, brute force, DoS, botnet.

TÀI LIỆU THAM KHẢO

- [1] M.-L. Dam et al., “Anomaly-based intrusion detection leveraging optimized firewall log analysis: a real-time machine learning solution,” *International Journal of Electrical and Computer Engineering*, vol. 15, no. 5, pp. 4785–4802, Oct. 2025, doi: 10.11591/ijece.v15i5.pp4785-4802.
- [2] The Zeek Project, “Zeek Network Security Monitor Documentation,” *Zeek.org*, 2024. [Online]. Available: <https://docs.zeek.org/en/master/>.
- [3] SecDev, “Scapy: Packet manipulation with Python,” *Scapy.net*, 2023. [Online]. Available: <https://scapy.net/>.
- [4] Microsoft, “LightGBM: A highly efficient gradient boosting decision tree,” *GitHub Repository*, 2024. [Online]. Available: <https://github.com/microsoft/LightGBM>.
- [5] Canadian Institute for Cybersecurity, “Intrusion Detection Evaluation Dataset (CIC-IDS2017),” *UNB.ca*, 2017. [Online]. Available: <https://www.unb.ca/cic/datasets/ids-2017.html>.
- [6] N. Moustafa and J. Slay, “UNSW-NB15: A comprehensive data set for network intrusion detection systems,” in *2015 Military Communications and Information Systems Conference (MilCIS)*, Canberra, Australia, 2015, pp. 1–6.
- [7] N. Koroniotis et al., “Towards the development of realistic botnet dataset in the Internet of Things for network forensic analytics: Bot-IoT dataset,” *Future Generation Computer Systems*, vol. 100, pp. 779–796, 2019.
- [8] F. Pedregosa et al., “Scikit-learn: Machine Learning in Python,” *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011. [Online]. Available: <https://scikit-learn.org/>.
- [9] Canonical Ltd., “Ubuntu Server 20.04 LTS Documentation,” *Ubuntu.com*, 2020. [Online]. Available: <https://ubuntu.com/server/docs>.
- [10] OffSec, “Kali Linux Documentation,” *Kali.org*, 2024. [Online]. Available: <https://www.kali.org/docs/>.